МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №7

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
Е.И. Мицкевич
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений.

Ход работы

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

• Реализовать соответствующие классы, указанные в задании;

• Организовать ввод параметров для создания объектов (использовать экранные компоненты);

• Осуществить визуализацию графических примитивов

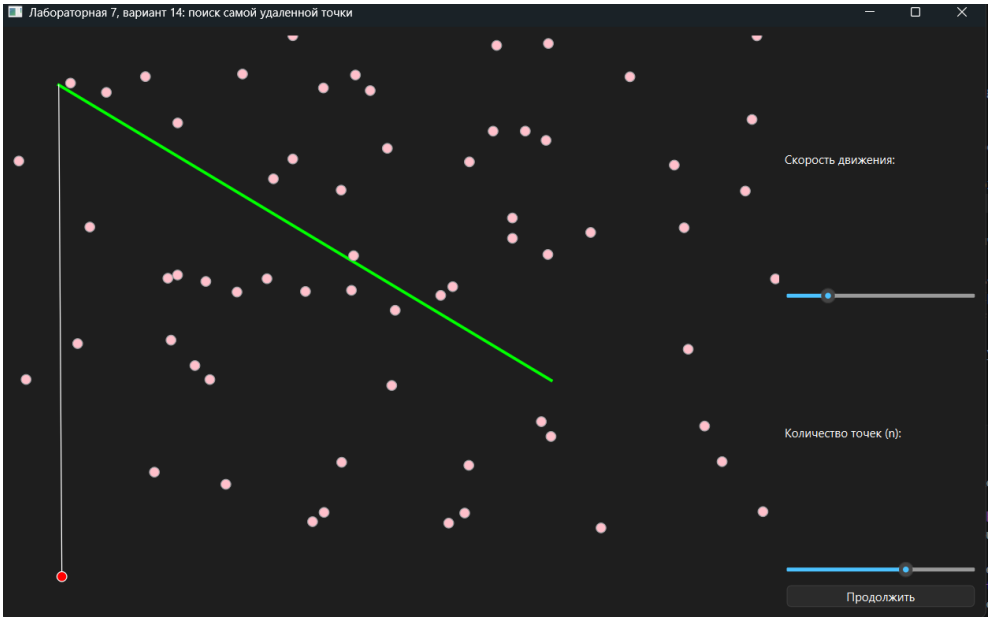
14) Создать классы Point и Line. Объявить список из n объектов класса Point и определить в методе, какая из точек находится дальше всех от прямой линии.

```
importsys
importrandom
importmath
importdatetime
fromPySide6.QtWidgets import(QApplication, QMainWindow, QWidget, QVBoxLayout,
QHBoxLayout, QPushButton, QSlider, QLabel) # строительные блоки
fromPySide6.QtCore importQTimer, Qt, QPointF # мозги системы, таймеры,
координаты, сигналы
fromPySide6.QtGui importQPainter, QColor, QPen # графика


# Класс точки
classPoint:
    def__init__(self, x, y):
        self.x=x
        self.y=y
        # Вектор движения
        self.vx=random.uniform(-1, 1)
        self.vy=random.uniform(-1, 1)


    defmove(self, width, height, speed):
        self.x+=self.vx*speed
        self.y+=self.vy*speed

        # Отскок от границ
        ifself.x<=0orself.x>=width: self.vx*=-1
        ifself.y<=0orself.y>=height: self.vy*=-1


# класс прямой
classLine:
    def__init__(self, p1: QPointF, p2: QPointF):
        self.p1=p1
        self.p2=p2


    defget_distance(self, pt: Point):
        # Формула расстояния от точки до прямой (x1, y1) - (x2, y2)
        x1=self.p1.x()
        y1=self.p1.y()
        x2=self.p2.x()
        y2=self.p2.y()

        numerator=abs((y2-y1) *pt.x-(x2-x1) *pt.y+x2*y1-y2*x1)
```

```
        denominator=math.sqrt((y2-y1)**2+(x2-x1)**2)
        returnnumerator/(denominator+0.0001) # защита от деления на 0


# Визуализация
classCanvas(QWidget):
    def__init__(self):
        super().__init__()
        self.points=[]
        # Создаем линию, передавая две точки (начало и конец) для расчетов
        self.line=Line(QPointF(50, 50), QPointF(550, 350))
        self.speed=1
        self.is_paused=False
        self.furthest_point=None# Переменная для хранения ссылки на самую
дальнюю точку


    defupdate_points_count(self, n):
        # Для каждой точки задаем случайные координаты x (от 10 до 750) и y (от
10 до 450)
        self.points=[Point(random.randint(10, 750), random.randint(10, 450))
for_inrange(n)]


    deffind_furthest(self):
        ifnotself.points: return
        # Поиск точки с максимальным расстоянием
        self.furthest_point=max(self.points, key=lambdap:
self.line.get_distance(p)) # сравниваем результаты метода get_distance


    # функция отрисовки
    defpaintEvent(self, event):
        painter=QPainter(self) # Создаем объект "Художник", который будет
рисовать на этом виджете
        painter.setRenderHint(QPainter.Antialiasing) # Включаем сглаживание,
чтобы линии были плавными

        # Рисуем линию
        pen_line=QPen(Qt.green, 3) # создаём перо толщина 3 пикселя
        painter.setPen(pen_line)
        painter.drawLine(self.line.p1, self.line.p2)


        # Рисуем точки
        self.find_furthest() # Перед отрисовкой вычисляем, какая точка сейчас
самая дальняя
        forptinself.points:
            ifpt==self.furthest_point: # если дальняя
                painter.setBrush(QColor("red"))# Красим точку внутри
                painter.setPen(QPen(Qt.white, 1))# обводка белая
                # Рисуем линию до дальней точки
                painter.drawLine(QPointF(pt.x, pt.y), self.line.p1)
            else:
                painter.setBrush(QColor("pink"))
                painter.setPen(QPen(Qt.gray, 1))

            # Рисуем саму точку как маленький круг (эллипс) радиусом 5 пикселей
            painter.drawEllipse(QPointF(pt.x, pt.y), 5, 5)


    # метод анимации
    defstep(self):
        ifnotself.is_paused:
            forptinself.points:
```

```
                pt.move(self.width(), self.height(), self.speed) # рассчёт
координат
            self.update()


classMainWindow(QMainWindow):
    def__init__(self):
        super().__init__()
        self.setWindowTitle("Лабораторная 7, вариант 14: поиск самой удаленной
точки")

        self.canvas=Canvas()
        self.canvas.update_points_count(10) # по умолчанию сначала 10 точек


        # складывает виджеты в столбик
        controls=QVBoxLayout()

        # Слайдер скорости
        controls.addWidget(QLabel("Скорость движения:"))
        self.speed_slider=QSlider(Qt.Horizontal)
        self.speed_slider.setRange(0, 10)
        self.speed_slider.setValue(2) # сначала такое
        self.speed_slider.valueChanged.connect(self.change_speed) # сигнал для
метода change_speed
        controls.addWidget(self.speed_slider) # добавляем в виджет


        # Слайдер количества точек
        controls.addWidget(QLabel("Количество точек (n):"))
        self.count_slider=QSlider(Qt.Horizontal)
        self.count_slider.setRange(2, 100)
        self.count_slider.setValue(10)
        self.count_slider.valueChanged.connect(self.change_count)
        controls.addWidget(self.count_slider)


        # Кнопки
        self.btn_pause=QPushButton("Пауза")
        self.btn_pause.clicked.connect(self.toggle_pause) # сигнал нажатия
кнопки
        controls.addWidget(self.btn_pause)


        # Сборка интерфейса
        layout=QHBoxLayout() # горизонтальный виджет
        layout.addWidget(self.canvas, stretch=4) # Добавляем холст. stretch=4
значит, что он заберет 80% ширины окна
        layout.addLayout(controls, stretch=1)  # Добавляем панель кнопок.
stretch=1 значит, что она заберет 20% ширины


        central_widget=QWidget()
        central_widget.setLayout(layout)
        self.setCentralWidget(central_widget)


        # QTimer — это бесконечный цикл, работающий в фоне
        self.timer=QTimer()

        self.timer.timeout.connect(self.canvas.step)
        self.timer.start(20)


    defchange_speed(self, value):
```

```
        self.canvas.speed=value


    defchange_count(self, value):
        self.canvas.update_points_count(value)


    deftoggle_pause(self):
        self.canvas.is_paused=notself.canvas.is_paused
        self.btn_pause.setText("Продолжить"ifself.canvas.is_pausedelse"Пауза")



if__name__=="__main__":
    app=QApplication(sys.argv)
    window=MainWindow()
    window.resize(1000, 600)
    window.show()
    sys.exit(app.exec())
```



Задание 2. Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

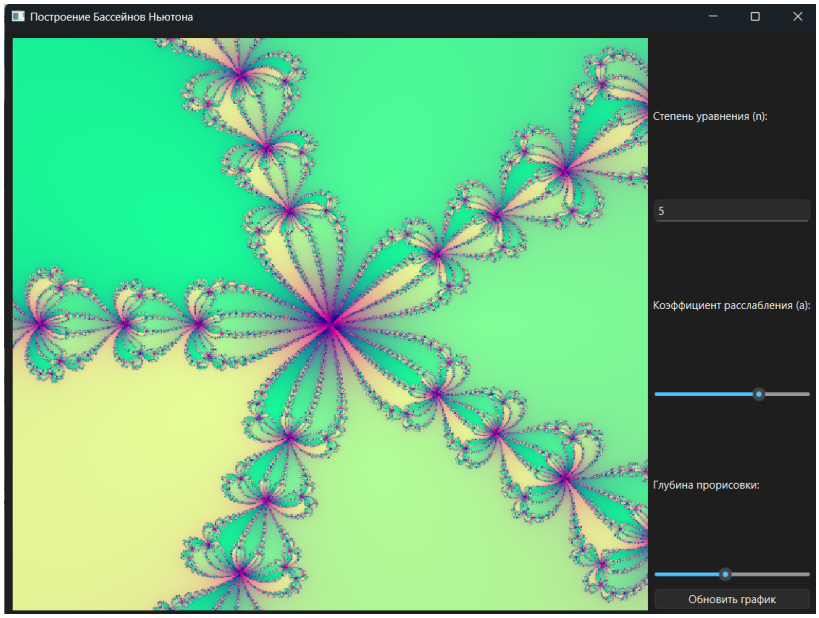
14) Бассейны Ньютона

```
importsys
importnumpyasnp
fromPySide6.QtWidgets import(QApplication, QMainWindow, QWidget, QVBoxLayout,
QHBoxLayout, QPushButton, QSlider, QLabel, QLineEdit)
fromPySide6.QtCore importQt
fromPySide6.QtGui importQImage, QPainter


#Класс логики фрактала
classNewtonFractal:
    def__init__(self, width, height):
        self.width=width
        self.height=height

    defgenerate(self, n, a, iterations):
```

```
        # Создаем координатную сетку от -2 до 2
        x=np.linspace(-2, 2, self.width)
        y=np.linspace(-2, 2, self.height)
        X, Y=np.meshgrid(x, y)
        Z=X+1j*Y# Превращаем в комплексную плоскость


        # Массив для фиксации скорости сходимости (для яркости)
        convergence=np.zeros(Z.shape, dtype=float)


        foriinrange(iterations):
            z_prev=Z.copy()
            # Формула Ньютона: z = z - a * (z^n - 1) / (n * z^(n-1))
            f_z=Z**n-1
            df_z=n*Z**(n-1)

            # Защита от деления на 0
            df_z[np.abs(df_z) <1e-6] =1e-6

            Z-=a*f_z/df_z

            # Считаем, насколько близко точка подошла к корню
            convergence+=np.exp(-np.abs(Z-z_prev))


        # Определяем корень через угол комплексного числа
        roots=np.angle(Z)

        # Подготовка данных для RGB картинки
        # Цвет (Hue) зависит от угла корня
        hue=((roots+np.pi) /(2*np.pi) *255).astype(np.uint8)
        # Яркость зависит от того, как быстро точка нашла корень
        brightness=((convergence/iterations) *255).astype(np.uint8)

        # Собираем финальный массив пикселей
        img_array=np.zeros((self.height, self.width, 3), dtype=np.uint8)
        img_array[..., 0] =hue        # Красный канал (зависит от корня)
        img_array[..., 1] =brightness # Зеленый канал (зависит от сходимости)
        img_array[..., 2] =150        # Синий канал (константа для красоты)

        returnimg_array


# Визуализация
classFractalCanvas(QWidget):
    def__init__(self):
        super().__init__()
        self.fractal_engine=NewtonFractal(600, 600)
        self.n=3
        self.a=1.0
        self.iters=30
        self.image=None
        self.render_fractal()


    defrender_fractal(self):
        # Генерируем массив данных фрактала
        raw_data=self.fractal_engine.generate(self.n, self.a, self.iters)
        h, w, _=raw_data.shape
        # Конвертируем массив NumPy в формат, понятный для Qt (QImage)
        self.image=QImage(raw_data.data, w, h, w*3, QImage.Format_RGB888)
        self.update() # Перерисовать виджет
```

```
    defpaintEvent(self, event):
        ifself.image:
            painter=QPainter(self)
            # Рисуем изображение, растягивая его на весь размер виджета
            painter.drawImage(self.rect(), self.image)


#Главное окно
classMainWindow(QMainWindow):
    def__init__(self):
        super().__init__()
        self.setWindowTitle("Построение Бассейнов Ньютона")

        self.canvas=FractalCanvas()

        # Панель параметров
        controls=QVBoxLayout()

        # Ввод степени n через текстовое поле
        controls.addWidget(QLabel("Степень уравнения (n):"))
        self.n_input=QLineEdit("3") # Начальное значение "3"
        self.n_input.setPlaceholderText("Введите число (напр. 3)")
        # Обновляем при нажатии Enter или потере фокуса
        self.n_input.editingFinished.connect(self.apply_params)
        controls.addWidget(self.n_input)


        # Слайдер коэффициента a (расслабление)
        controls.addWidget(QLabel("Коэффициент расслабления (a):"))
        self.a_slider=QSlider(Qt.Horizontal)
        self.a_slider.setRange(1, 20)
        self.a_slider.setValue(10) # соответствует 1.0
        self.a_slider.valueChanged.connect(self.apply_params)
        controls.addWidget(self.a_slider)


        # Слайдер итераций (качество)
        controls.addWidget(QLabel("Глубина прорисовки:"))
        self.iter_slider=QSlider(Qt.Horizontal)
        self.iter_slider.setRange(5, 60)
        self.iter_slider.setValue(30)
        self.iter_slider.valueChanged.connect(self.apply_params)
        controls.addWidget(self.iter_slider)


        # Кнопка принудительного обновления
        self.btn_refresh=QPushButton("Обновить график")
        self.btn_refresh.clicked.connect(self.apply_params)
        controls.addWidget(self.btn_refresh)

        # Располагаем всё в окне
        layout=QHBoxLayout()
        layout.addWidget(self.canvas, stretch=4)
        layout.addLayout(controls, stretch=1)

        central_widget=QWidget()
        central_widget.setLayout(layout)
        self.setCentralWidget(central_widget)


    defapply_params(self):
        # Считываем данные из текстового поля с обработкой ошибок
        try:
            val_n=float(self.n_input.text())
            ifval_n==0: val_n=0.1# Защита от нулевой степени
```

```
            self.canvas.n=val_n
        exceptValueError:
            # Если ввели буквы, просто не меняем значение n
            pass


        # Считываем слайдеры
        self.canvas.a=self.a_slider.value() /10.0
        self.canvas.iters=self.iter_slider.value()

        # Перерисовываем фрактал
        self.canvas.render_fractal()


if__name__=="__main__":
    app=QApplication(sys.argv)
    window=MainWindow()
    window.resize(900, 650)
    window.show()
    sys.exit(app.exec())
```



Вывод: были освоены возможности языка программирования Python в разработке оконных приложений.